Design and Implementation of a

Proximal Policy Optimisation Controller for a

Non-linear Quadrotor Under Stochastic Disturbances

Henry Rochester

Supervisor: Dr Malcolm Hillebrand

Department of Mathematics and Applied Mathematics
University of Cape Town

May 24, 2026

Introduction

We now implement a Proximal Policy Optimisation (PPO) controller for the quadrotor. The objective of this controller is to learn a feedback policy which maps the current 12-dimensional state of the quadrotor directly to a 4-dimensional control input. That is, instead of explicitly deriving a control law from the dynamics, as in the case of LQR, or manually tuning our error feedback gains, as we did in the case of PID, we allow the controller to learn a suitable control strategy through repeated interaction with the hover environment.

Nonlinear Quadrotor State-Space Model

Recall that the quadrotor state vector is defined as:

$$\vec{x} = \begin{bmatrix} x \\ y \\ z \\ v_x \\ v_y \\ v_z \\ \phi \\ \theta \\ \psi \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}.$$

The control input is given by:

$$\vec{u} = \begin{bmatrix} T & \tau_x & \tau_y & \tau_z \end{bmatrix}^T,$$

where T is the total thrust, and τ_x, τ_y, τ_z are the body torques about the roll, pitch, and yaw axes respectively. The aim is for PPO to learn the control actions required to stabilise the quadrotor near the desired hover target.

In the PPO setting, the controller does not explicitly compute this input from a derived feedback gain matrix, as in the case of LQR. Instead, the policy network learns a function of the form:

$$\pi_\theta : \mathbb{R}^{12} \to \mathbb{R}^4,$$

where θ represents the parameters of the neural network. Thus, given the current state observation, the policy network outputs the thrust and torque commands to be applied to the quadrotor.

Proximal Policy Optimisation

Before describing the implementation details, it is useful to explain what Proximal Policy Optimisation (PPO) is and how it works.

PPO is a reinforcement learning algorithm used to train an agent to make sequential decisions. In this case, the agent is our quadrotor controller. At each timestep, the controller observes the current 12-dimensional state of the drone and chooses an action, namely the thrust and torques to apply. The environment then evolves according to the quadrotor dynamics within our designed hover environment and returns a reward which measures how good or poor the chosen action was.

The objective of PPO is therefore to learn a function, which we call a policy, denoted by π_θ, which chooses actions that maximise the total expected return, G_t, over an episode, via the clipped objective function $L^{\text{CLIP}}(\theta)$. This return is determined by the sequence of rewards r_t, which are defined later. In our case, this means learning actions which drive the drone towards its target position. With regards to stabilising the drone about its hover position, the policy must learn to reduce velocity and angular motion, and avoid unstable behaviour such as excessive tilt or falling away from the target.

PPO is known as a policy-gradient method. That is to say that, instead of the controller first learning an explicit model for the nonlinear dynamics of the environment, the controller learns the policy directly:

$$\pi_\theta(\vec{u}_t \mid \vec{x}_t),$$

where θ denotes the parameters of the neural network.

Now, we note that the expression $\pi_\theta(\vec{u}_t \mid \vec{x}_t)$ denotes the probability distribution over actions given the current state at time t. During training, the agent, which in our case is the controller, samples different possible thrust and command torques in order to determine which actions produce the best possible reward. This leads us to the actor-critic structure of PPO:

Actor-Critic Structure

PPO uses an actor-critic framework. This means that two related functions, namely the actor network and the critic network, are learned during training.

The actor network is the policy. It is responsible for selecting the action:

$$\vec{u}_t \sim \pi_\theta(\cdot \mid \vec{x}_t).$$

In our case, the actor receives the 12-dimensional quadrotor state and outputs the 4-dimensional control action. Thus, the actor forms the part of the PPO algorithm which eventually becomes the controller.

The critic network estimates the value function:

$$V_\phi(\vec{x}_t) = \mathbb{E}_{\pi_\theta}\left[G_t \mid \vec{X}_t = \vec{x}_t\right],$$

where ϕ denotes the critic parameters. The value function estimates how good the current state is, when following the current policy π_θ, in terms of the future return expected from the current state. The return G_t is defined below. This is how the controller is able to determine which thrust and command torque combination is best, given the possible state the drone is in at time t.

For example, if the drone is close to the target, has low velocity, and has small attitude angles, then the critic should assign that state a relatively high value. If the drone is tilted aggressively, moving away from the target, or close to flipping or crashing, then the critic should assign that state a lower value.

Reward and Return

At each timestep, the environment returns a reward r_t. The reward function in reinforcement learning is designed to encode the control objective. We define our reward function as follows:

$$r_t = 0.1 - \left(w_{\text{pos}} \left\|\vec{p}_t - \vec{p}^*\right\|^2 + w_{\text{vel}} \left\|\vec{v}_t\right\|^2 + w_{\text{angle}} \left(\phi_t^2 + \theta_t^2\right) + w_\omega \left\|\vec{\omega}_t\right\|^2 + w_{\text{eff}} \left\|\vec{u}_t - \vec{u}_{\text{hov}}\right\|^2\right).$$

Here, w_{pos}, w_{vel}, w_{angle}, w_ω, and w_{eff} are the position, velocity, Euler angle, angular velocity, and efficiency weights, respectively. These are hyperparameters which are yet to be fine-tuned.

The intuition behind this reward shape is to penalise large position error, high veloc-

ity, excessive attitude angles, large angular velocity, and unstable control actions, such as aggressive changes in thrust, or body torques.

The total discounted return from time t is given by:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots ,$$

where $\gamma \in [0, 1]$ is the discount factor.

The purpose of the discount factor is to control how much the agent values future rewards. In this implementation, we use:

$$\gamma = 0.99.$$

This means the controller is encouraged to care about long-term stability, rather than only maximising the immediate reward at the next timestep, since higher powers of γ are still meaningful. This is important for our quadrotor control problem because an aggressive action, such as exerting an immediate high thrust value, may temporarily reduce position error while causing high velocity or angular instability later.

The Advantage Function

We now consider how PPO updates the policy, π_θ. We note that it does not update the policy using the raw return alone. Instead, it uses what we call the advantage function, A_t. The advantage measures whether an action was better or worse than expected from that state, when following the current policy π_θ. It is commonly written as:

$$A_t = G_t - V_\phi(\vec{x}_t).$$

If $A_t > 0$, then the action taken was better than the critic network expected. PPO should therefore increase the probability of taking similar actions in similar states. If $A_t < 0$, then the action was worse than expected, and PPO should reduce the probability of taking similar actions.

The Clipped Objective Function

A notable idea behind the PPO algorithm is that we want the policy to improve, but we do not want the policy parameters to change too much within a single update. The reason for this is that, if the policy changes too aggressively after one training update, it may suddenly become unstable despite previously producing stable behaviour. In the

context of our quadrotor control problem, this could mean that a controller which was previously keeping the drone stable suddenly applies excessive thrust or torques at the next timestep, causing the drone to become unstable. This is precisely the type of behaviour we want to avoid.

To prevent this, PPO compares the new policy, computed during the current update step, with the old policy from the previous update step. This is done using the following probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(\vec{u}_t \mid \vec{x}_t)}{\pi_{\theta_{\text{old}}}(\vec{u}_t \mid \vec{x}_t)}.$$

This ratio measures how much more or less likely the new policy is to take the same action compared to the old policy. If $r_t(\theta)$ is close to 1, then the new policy is similar to the old policy. If $r_t(\theta)$ is much larger or much smaller than 1, then the policy has changed significantly.

PPO then uses the following clipped objective function:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) A_t \right) \right].$$

In this implementation, we use:

$$\epsilon = 0.2,$$

which is the recommended value used by Schulman et al. (2017).

The effect of this objective is that PPO discourages the new policy from moving too far away from the old policy during a single update. The policy is still allowed to improve, but the clipping mechanism prevents overly large and potentially destructive updates. This makes PPO particularly useful for our quadrotor control problem, where sudden changes in the controller may result in unstable flight behaviour.

On-Policy Training

A final notable point about the PPO algorithm is that it is an on-policy algorithm. This means that the data used to update the policy must be collected using the current version of the policy. Once the policy has been updated, the old rollout data becomes less useful, because it was generated by an older version of the policy, and is therefore discarded.

This is different from off-policy algorithms, which can store old transitions in a replay buffer and reuse them many times. PPO does not work in this way. Instead, PPO collects fresh rollouts using the current policy, updates the policy for a limited number of epochs, and then collects new data using the updated policy.

We note that in practice, we split the rollout data into mini-batches during training. PPO then makes several update passes over these mini-batches in order to extract as much useful information as possible from the collected data, without moving the policy too far away from the policy which generated that data in the first place. After these update steps, the old rollout data is discarded and the process repeats.

For our quadrotor controller, this means that the controller is always learning from data generated by its most recent behaviour. As the policy improves, the drone explores new regions of the state space, and PPO continues to update the controller based on these fresh interactions with our hover environment.

PPO Training Loop

We can summarise the PPO training process as follows:

1. We initialise the actor and critic neural networks;
2. Run the current policy in the quadrotor environment for a fixed number of timesteps;
3. Store the observations, actions, rewards, value estimates, and log probabilities, for numerical stability;
4. Use the rewards and critic estimates to compute advantage estimates;
5. Split the collected rollout data into mini-batches;
6. Update the actor using the PPO clipped objective;
7. Update the critic by reducing the error between predicted values and observed returns;
8. Repeat the process with the newly updated policy.

In our implementation, this process is handled by calling the Stable-Baselines3 call:

```
model.learn(total_timesteps=total_timesteps, callback=eval_callback)
```

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal Policy Optimization Algorithms*, 2017. Available at: `https://arxiv.org/abs/1707.06347`

[2] Stable-Baselines3, *PPO*. Stable-Baselines3 Documentation. Available at: https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html#notes.

[3] S. V. Albrecht, F. Christianos, and L. Schäfer, *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. Available at: https://www.marl-book.com.

[4] Arxiv Insights, *An Introduction to Policy Gradient Methods – Deep Reinforcement Learning*. YouTube, 2018. Available at: https://www.youtube.com/watch?v=5P7I-xPq8u8.

[5] A. Raffin et al., *Stable-Baselines3: Reliable Reinforcement Learning Implementations*, Journal of Machine Learning Research, 2021. Available at: https://jmlr.org/papers/v22/20-1364.html

[6] Farama Foundation, *Gymnasium Documentation*. Available at: https://gymnasium.farama.org/

[7] OpenAI, *Spinning Up in Deep Reinforcement Learning: Proximal Policy Optimization*. Available at: https://spinningup.openai.com/en/latest/algorithms/ppo.html